Orbit Mania: A Newtonian Orbital Simulation and Reinforcement Learning Environment

Adriel I. Santoso
Department of Mechanical and Aerospace Engineering
Tohoku University

April 3, 2026

Abstract

Orbit Mania is a dual-purpose software artifact combining a physics-accurate simulation of Keplerian orbital dynamics with a reinforcement learning (RL) environment conforming to the OpenAI Gymnasium standard. The simulation models spacecraft motion using polar-coordinate kinematics and a stylised Hohmann transfer mechanic. An autonomous agent trained via Proximal Policy Optimisation (PPO) navigates the same asteroid-evasion task presented to human players, enabling direct comparison of human and machine performance under resource constraints. This report describes the theoretical foundations, system architecture, simulation engine, RL environment specification, and training procedure.

1 Introduction

Orbit Mania is a project at the intersection of arcade game design, classical orbital mechanics, and machine learning. Originally conceived as a simple Pygame game in which a spacecraft evades asteroids while managing fuel, it has been substantially elevated into a dual-purpose artifact: a **physics-accurate simulation** of Keplerian orbital dynamics and a **reinforcement learning (RL) environment** conforming to the OpenAI Gymnasium standard.

The project pursues two complementary scientific goals:

1. To model orbital motion using polar-coordinate kinematics and smooth radius interpolation, producing physically intuitive spacecraft behaviour.

2. To train an autonomous agent using the Proximal Policy Optimisation (PPO) algorithm to survive the asteroid field, providing a benchmark for comparing human and machine decision-making under time pressure and resource constraints.

The concept was motivated by the **Hohmann transfer**—a space manoeuvre encountered in the study of orbital mechanics that enables energy-efficient transitions between circular orbits. This transfer serves as the principal mechanic by which both human players and the AI agent switch between the two discrete orbital radii available in the environment.

2 Theoretical Background

2.1 Polar Representation of Circular Orbits

The spacecraft's position is computed at each simulation tick using a polar-to-Cartesian transformation. The position vector $\mathbf{p}$ is given by

$$\mathbf{p}(\theta) = \begin{bmatrix} r\cos\theta \\ r\sin\theta \end{bmatrix} \tag{1}$$

where r is the instantaneous orbital radius and θ is the current angular position, incremented each frame by a fixed angular speed $\dot{\theta} = 0.8$ degrees per tick.

2.2 Circular Orbital Speed

A stable circular orbit occurs when gravitational attraction exactly balances centripetal acceleration:

$$\frac{GMm}{r^2} = \frac{mv^2}{r} \quad \Longrightarrow \quad v = \sqrt{\frac{GM}{r}} \tag{2}$$

In the simulation the product GM is a dimensionless constant tuned to give visually plausible motion ($GM = 1000$). The two available orbital radii are $r_1 = 125\,\text{px}$ (inner) and $r_2 = 225\,\text{px}$ (outer), yielding circular speeds of approximately $v_1 \approx 2.83\,\text{px/tick}$ and $v_2 \approx 2.11\,\text{px/tick}$ respectively.

2.3 The Hohmann Transfer

The Hohmann transfer is an orbital manoeuvre that transitions a spacecraft between two coplanar circular orbits using exactly two engine burns. The first burn accelerates the craft into an elliptical transfer orbit whose periapsis lies on the inner orbit and apoapsis on the outer; the second burn circularises the trajectory at the target radius.

In the game, this transfer is stylised: pressing SPACE sets a *target radius* that the current radius interpolates towards at a fixed rate of 0.44 px/tick, producing the characteristic smooth spiral animation. This approximation preserves the qualitative character of the Hohmann transfer without requiring a full Keplerian elliptic integration.

2.4 Reinforcement Learning

Reinforcement learning frames interaction as an agent–environment loop. At each discrete time step t, the environment exposes a **state observation** $s_t \in \mathcal{S}$ to the agent. The agent selects an **action** $a_t \in \mathcal{A}$ according to its policy $\pi(a_t \mid s_t)$, and receives a scalar **reward** r_t from the environment. The agent's objective is to maximise the expected discounted return:

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k} \tag{3}$$

where $\gamma \in [0, 1)$ is a discount factor.

Proximal Policy Optimisation

PPO [1] is a policy-gradient algorithm that alternates between sampling trajectories from the current policy and performing multiple stochastic gradient-ascent steps on a clipped surrogate objective:

$$\mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E}_t\Big[\min\Big(r_t(\theta)\hat{A}_t,\ \text{clip}(r_t(\theta), 1-\varepsilon, 1+\varepsilon)\,\hat{A}_t\Big)\Big] \tag{4}$$

where $r_t(\theta) = \pi_\theta(a_t|s_t)/\pi_{\theta_{\text{old}}}(a_t|s_t)$ is the probability ratio and $\hat{A}_t$ is a generalised advantage estimate. The clipping mechanism prevents excessively large policy updates, ensuring stable learning.

3 System Architecture

The project is organised into four Python source files, each with a distinct responsibility, shown in Figure 1.

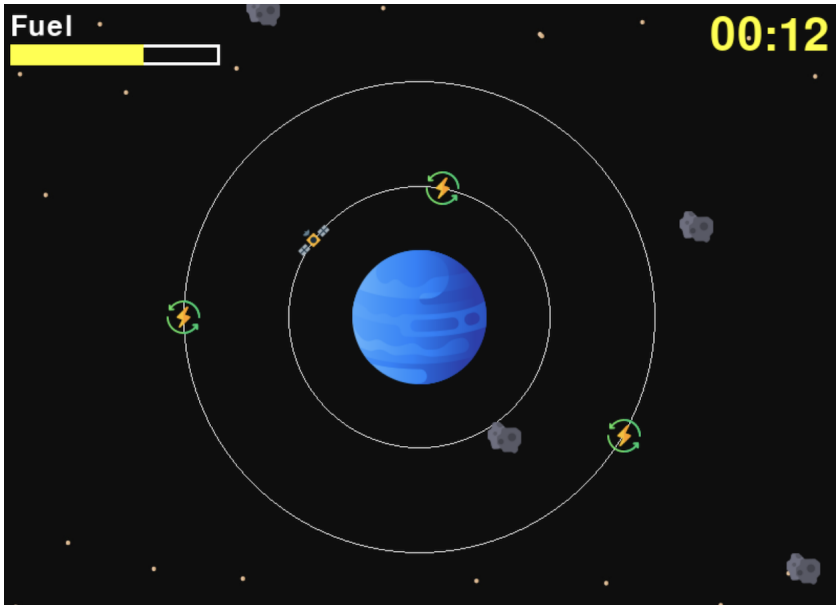

Figure 1: In-game screenshot showing the spacecraft on the inner orbit, energy packets on both orbits, and incoming asteroids.

File	Responsibility
`env.py`	Core simulation logic and Gymnasium interface. No graphics.
`play.py`	Pygame rendering loop; bridges `env.py` to the screen.
`train.py`	Headless PPO training pipeline using Stable-Baselines3.
`utilities.py`	Asset loader (sprites, sounds, fonts) for Pygame.

The critical design principle is **decoupling**: `env.py` runs with no graphical dependency. This allows `train.py` to spin up thousands of simulation ticks per second (CPU-bound only), accumulating experience far faster than real-time rendering would permit.

4 The Simulation Engine

4.1 State Variables

Each episode maintains the following state:

- θ (`angle`): current angular position in degrees, incremented each tick.

- r (`radius`): instantaneous orbital radius in pixels.
- r^* (`target_radius`): target radius; either $r_1 = 125$ or $r_2 = 225$.
- f (`fuel`): scalar in $[0, 100]$ representing energy reserves.
- σ (`shield_active`): Boolean; whether the deflector shield is active.
- $\mathcal{O}$: list of active obstacle dictionaries (position, velocity, radius).
- $\mathcal{E}$: list of active energy-packet dictionaries (position, radius).

4.2 Simulation Tick

Each call to `step(action)` performs the following operations in order:

1. **Orbit switch**: if `action[0] = 1` and the ship is settled ($|r - r^*| < 1$), toggle r^*.
2. **Radius lerp**: move r towards r^* by at most $\Delta r = 0.44$ px.
3. **Angle advance**: $\theta \leftarrow \theta + 0.8$.
4. **Position update**: $\mathbf{p} = (r\cos\theta,\ r\sin\theta)$.
5. **Fuel drain**: $f \leftarrow f - d$, where $d = 0.2$ (shield on) or 0.05 (off).
6. **Obstacle spawn**: spawn at scheduled interval (difficulty-dependent).
7. **Energy packet spawn**: spawn if $|\mathcal{E}| < 5$ and interval elapsed.
8. **Obstacle movement**: each obstacle translates leftward at its fixed speed.
9. **Culling**: remove obstacles whose x-coordinate falls below -1000.
10. **Collision detection**: check ship against obstacles and energy packets.
11. **Termination check**: end episode if collision or fuel depleted.

4.3 Difficulty Scaling

A *difficulty scalar* $d \in [0, 1]$ increases with the step count:

$$d = \min\left(1,\ \frac{t}{3600}\right) \tag{5}$$

The obstacle spawn interval I and speed range $[s_{\min}, s_{\max}]$ are then interpolated:

$$I = I_0 - d\,(I_0 - I_{\min}) \tag{6}$$

$$s \sim \mathcal{U}\big[s_0^{\min} + d\,\Delta s^{\min},\ s_0^{\max} + d\,\Delta s^{\max}\big] \tag{7}$$

	$t = 0$	$t = 30\,\mathrm{s}$	$t \geq 60\,\mathrm{s}$
Spawn interval (ticks)	180	112	45
Speed range (px/tick)	1.0–2.0	1.75–3.5	2.5–5.0

5 The Reinforcement Learning Environment

5.1 Observation Space

The agent receives an 11-dimensional continuous observation vector $\mathbf{o} \in \mathbb{R}^{11}$ at each step:

Index	Feature	Range	Notes
0	$\sin\theta$	$[-1, 1]$	Encodes angular position without wrap
1	$\cos\theta$	$[-1, 1]$	Paired with $\sin\theta$ for full 2π coverage
2	$(r - r_1)/(r_2 - r_1)$	$[0, 1]$	Normalised orbital radius
3	$f/f_{\max}$	$[0, 1]$	Normalised fuel level
4	σ	$\{0, 1\}$	Shield state
5–6	$\mathbf{p}_{\mathrm{ob}} - \mathbf{p}$	$\mathbb{R}^2$	Nearest obstacle relative position
7–8	$\mathbf{v}_{\mathrm{ob}}$	$\mathbb{R}^2$	Nearest obstacle velocity
9–10	$\mathbf{p}_{\mathrm{ep}} - \mathbf{p}$	$\mathbb{R}^2$	Nearest energy packet relative position

Encoding the angle as $(\sin\theta, \cos\theta)$ rather than as a raw degree value avoids the discontinuity at 0/360 and keeps all inputs bounded.

5.2 Action Space

The action space is `MultiDiscrete([2, 2])`, decomposed as:

Dimension	Meaning
`action[0]`	Orbit switch: 0 = no-op, 1 = request switch
`action[1]`	Shield: 0 = off, 1 = on

5.3 Reward Function

The reward r_t at each time step is composed of:

$$r_t = r_{\text{alive}} + r_{\text{fuel}} + r_{\text{shield}} + r_{\text{terminal}} \tag{8}$$

Event	Reward	Notes
Surviving one step	$+0.1$	Encourages longevity
Collecting energy packet	$+20$	Incentivises fuel management
Obstacle destroyed by shield	$+5$	Rewards active shield use
Collision (no shield)	-100	Terminal penalty
Fuel depletion	-20	Terminal penalty

5.4 Training Procedure

The PPO agent is trained using Stable-Baselines3 with an MLP policy network. Training runs for 10^5 timesteps by default, with evaluation checkpoints every 10,000 steps. The best-performing model is saved to `models/ppo_orbit_mania.zip` and TensorBoard logs are written to `logs/`.

Listing 1: Launching the AI agent in human-observable mode.

```
python src/play.py --ai --model models/ppo_orbit_mania.zip
```

6 Human Interaction Mode

6.1 Controls

Key	Action
SPACE	Request orbit switch (smooth animated transition)
`S` (hold)	Activate deflector shield
ESC	Quit

6.2 HUD Elements

The heads-up display overlaid on the Pygame window provides:

- **Fuel bar**: yellow fill over a white border; depletes over time.
- **Orbit readout**: current instantaneous radius in pixels.
- **Elapsed time**: formatted MM:SS stopwatch in the top-right corner.
- **Control hint**: reminder of key bindings at the bottom of the screen.

7 Software Dependencies

Package	Purpose
`pygame`	2D rendering, input handling, audio
`numpy`	Array operations for position and velocity vectors
`gymnasium`	Standard RL environment interface
`stable-baselines3`	PPO algorithm implementation
`tensorboard`	Training metrics visualisation

Install all dependencies via:

Listing 2: Installing project dependencies.

```
pip install -r requirements.txt
```

8 Conclusion

Orbit Mania demonstrates that a small, well-structured codebase can serve simultaneously as an interactive game, a physics demonstration, and a rigorous RL testbed. By decoupling the simulation engine from the graphical renderer, the environment supports both real-time human play and high-throughput headless training without modification.

Future directions include: (1) replacing the stylised radius-lerp with a true Keplerian elliptic orbit integration; (2) introducing multi-objective reward shaping to study fuel-efficiency vs. survival trade-offs; (3) benchmarking multiple RL algorithms (e.g., DQN, A2C) against the PPO baseline; and (4) adding a comparative analysis of human versus agent performance as a function of difficulty level.

References

[1] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov, *Proximal Policy Optimization Algorithms*, arXiv:1707.06347, 2017.

[2] M. Towers *et al.*, *Gymnasium*, Farama Foundation, 2023. `https://gymnasium.farama.org`

[3] A. Raffin *et al.*, *Stable-Baselines3: Reliable Reinforcement Learning Implementations*, *Journal of Machine Learning Research*, 22(268):1–8, 2021.